

CS 354 Fall 2025

Lab 8: Xinu File System Structure [20 pts]

Due: 11/15/2025 (Saturday), 11:59 PM

Objectives

You will understand how the Xinu file system stores files on a disk with fixed-size blocks. You will see how the file system keeps metadata (e.g., the name of a file, the file size, and the list of disk blocks that contain the file contents).

Readings

Read Chapter 19 in the text and review module 19 of the slides. Look at the data structures in `include/filesys.h` that define the contents of a directory and the index blocks used for a Xinu file system. Also note the layout of the file system on a disk and the layout of index blocks. Observe the interpretation of the third argument for `read()` when reading a block from a disk device. Finally, look at function `lfibget` which reads an index block from disk.

Source code

Start from a fresh copy of XINU, `/homes/cs354/CS354_lab8.tgz`. To prevent running out of space, remove old copies of XINU (at least the cross-compiler) from your account. Then `untar CS354_lab8.tgz` to get a directory named `xinu-bbb-lab8` that contains a version of XINU you will use for this lab.

Steps You Will Take

0. Listen to the TAs give hints about the lab.

1. Create a function `myfcreate` with the following function signature and place it in `system/myfcreate.c`.

```
void myfcreate(  
    did32 filesystem,      /* Device ID of the local file system */  
    char *filename,        /* The name of a file to create      */  
    char *data,            /* Data to be written to the file    */  
    int32 size)            /* The size of the file to create    */
```

Your code should check arguments, where possible.

The first argument is the device ID of the local file system. You will use `LFILESYS` as the file system device in this lab). The second argument is a null-terminated name for the file to be created.

The third argument is a null-terminated string of characters to place in the file. `myfcreate` repeats characters from the string as needed to make a file of the specified size. For example, consider the C string "abc". If the size is 7, the file will contain seven bytes with the values: 'a', 'b', 'c', 'a', 'b', 'c', 'a'.

The fourth argument gives the size of the file. A negative size is invalid.

2. Create a function `mylfdump` with the signature, `void mylfdump(did32 diskdev)`, and place it in `system/mylfdump.c`.

This function lists the files on the disk, lists the index blocks for the file and for each index block, lists data block numbers. That is, you will dump the metadata associated with a file.

The output begins with a line that specifies the number of files

```
The directory contains XX files
```

After the heading line, the output contains information for each file. For example, if the file system only contains a file named `abc` that has 8999 bytes of data and a file named `def` that has 1 byte of data, the output should appear in the following form:

```

The directory contains 2 files
* File abc contains 8999 bytes of data
iblock 2 points to data blocks 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26
iblock 4 points to data blocks 27 28
* File def contains 1 bytes of data
iblock 5 points to data blocks 30

```

where each line lists an index block for the file followed by all the data block numbers in the index block. If the directory had more files the output for each file would appear. Similarly, if a file had more iblocks, they would all be listed.

3. Insert a set of calls to `myfcreate` immediately following the call to `lfscreate` (i.e., before the invocation of the shell in `main.c`). Have the calls create the following files (Note: The quotation marks aren't written to disk):

File Name	Content String	Total Size (bytes)
file1	" Hello "	500
file2	"--X--"	600
file3	"CS354 is fun."	9000
file4	"* * * * * "	17000
file5	"In a Xinu file..."	170
file6	"tee-hee "	40

4. After creating the files, allow the Xinu shell to run. Enter the command

```
ls -l
```

and verify that the six files are present and their sizes are correct.
5. Use the `cat` command to check the contents of each file. Note that the files do not end with a NEWLINE. So, entering an extra NEWLINE after each `cat` finished will move to a new line for the next `cat` command. You should at least confirm the specified string of characters has been replicated.

```

cat file1
cat file2
cat file3
cat file4

```

```
cat file5
cat file6
```

6. Place a call to `mylfdump(Lf_data.lf_dskdev)` immediately after the last call to `myfcreate`. The idea is to dump the disk and see the metadata for the six files above. You should see the dump output before the shell header. Answer question 5.
7. Once you know that `myfcreate` works correctly, modify `main.c` to print a dump of the file system each time the shell completes. To print such a dump, insert a call to `mylfdump(Lf_data.lf_dskdev)` after the `if` statement that calls `receive()` (i.e., just before the `kprintf` about recreating the shell). Run the resulting version of Xinu, and type `control-d` to exit the shell. You should see a dump of the file system and then the shell will run again.
8. Run your version of Xinu again. When the shell prompts, enter

```
date > file4
```

and then

```
cat file4
```

and check that a `date` string has replaced the previous contents of the file. Exit the shell. To do so, enter

```
control-d.
```

Look at the new dump and answer question 6. The shell will have restarted.

Enter:

```
echo Hello there > file9
control-d
```

Look at the new dump and answer question 7.

9. Continue with the above step, and enter:

```
echo +++ hi there > a
cat a a a a a a a a > b
```

Then, to see that the results are correct, enter

```
cat b
```

To exit the shell, press the following on your keyboard

```
control-d
```

Look at the new dump and answer question 8.

Questions

Place **short answers** to the following questions in a file named `questions.pdf`. In the case of using document software (e.g., Word), please make sure to convert the file to **PDF**.

1. The function `myfcreate` takes a file system device ID as an argument. Would it be possible to take an arbitrary `*name*` of a file system device instead? For example, instead of using `LFILESYS`, could you rewrite the function so a caller could enter `"LFILESYS"`? Explain. (Hint: does Xinu store device names?)
 2. Suppose you wanted to extend the `dump` program to count blocks on the data block free list. Briefly sketch how you would do it or explain why it is impossible.
 3. If file system code contains an error, the file system might be left in an inconsistent state. For example, if the file system fails to put an iblock back on the free list when deleting a file, the iblock would not be in any file and would not be on the free list. We use the term “orphan” to describe such an iblock. Programs such as Unix's `fsck` check the Unix file system for and handle orphan inodes. If it is possible write a Xinu function to put orphan iblocks back on the free list, briefly explain the algorithm you would use. If such a function cannot be written, explain why.
 4. Consider writing a function that checks the data blocks in a Xinu file system to find orphaned data blocks that are neither part of a file nor on the free list. You may assume that the directory and iblocks in the file system are correct. If it is possible to write a function that repairs orphaned data blocks, briefly explain the algorithm you would use. If such a function cannot be written, explain why.
 5. According to your dump in Step 6, when `file4` was first created, which iblocks and data blocks were used?
 6. You stored the date into `file4` in Step 8, which iblocks and data blocks are used for the file? Explain any changes from the previous question.
 7. In step 8, after you create `file9`, which iblocks and data blocks does it use? Explain.
 8. After you exit the shell in Step 9, how many data blocks does file `b` contain? Explain.
-

Turn-in instructions

General instructions:

When implementing code in the labs, please maintain separate versions/copies of code so that mistakes such as unintentional overwriting or deletion of code is prevented. This is in addition to the efficiency that such organization provides. You may use any number of version control systems such as GIT and RCS. Please make sure that your code is protected from public access. For example, when using GIT, use git that manages code locally instead of its on-line counterpart github. If you prefer not to use version control tools, you may just use manual copy to keep track of different versions required for development and testing. More vigilance and discipline may be required when doing so.

Submission format for Lab 8. **Make sure to “make clean” before submission and omit the cross_compiler folder.**

lab8

|--- xinu-bbb-lab8

|----- compile

|----- ...

|----- config

|----- ...

|----- DESCRIPTION

|----- device

|----- ...

|----- include

|----- ...

|----- lib

|----- ...

```
|----- net
|----- ...
|----- shell
|----- ...
|----- system
|----- myfcreate.c
|----- mylfdump.c
|----- main.c
|----- ...
|--- questions.pdf
```

Electronic turn-in instructions:

- 1) Please make sure to follow the file and function naming convention as mentioned in the lab handout.
- 2) Please make sure your code does not depend on the contents of `main.c`, we will replace this during testing
- 3) Go to the `xinu-bbb-lab8/compile` directory and run: `make clean`.
- 4) Go to the directory where `lab8` (lab directory containing `xinu-bbb-lab8/` and `questions.pdf`) is a subdirectory.
 - a) For example, if
`/homes/alice/my_cs354_stuff/lab8/xinu-bbb-lab8` is your directory structure use the following commands to submit your lab
- 5) Type the following command

```
cd /homes/alice/my_cs354_stuff
turnin -c cs354 -p lab8 lab8
```

You can check/list the submitted files using

```
turnin -c cs354 -p lab8 -v
```

Please make sure to disable all debugging output before submitting your code.

